How OperatorOS Works

The Problem We Are Trying to Solve

Learning how to operate an unfamiliar manufacturing machine can be difficult before a student ever reaches the machine shop.

Training programs do not always have enough introductory material to teach students what a machine is, what its major parts are, what each part does, and which parts deserve extra attention. As a result, a student may meet the machine and the technician at the same time. The technician must then spend valuable in-person training time explaining basic information before the student can begin learning how to operate the machine.

This creates several problems:

1. Students begin hands-on training without enough background knowledge.
2. Technicians spend time repeatedly explaining the same basic machine components.
3. Important safety details may be difficult to remember when they are presented only verbally.
4. User manuals are often long, technical, and difficult to read under time pressure.
5. Diagrams and machine-specific terms can be hard to understand without seeing the actual machine.
6. A student may not know what questions to ask until they are already standing in front of the equipment.

OperatorOS is intended to help students frontload this learning. It gives them a way to explore a machine through video, ask questions about what they see, consult relevant manual information, and visually identify or track parts before they begin in-person training.

OperatorOS is not intended to replace a trained technician. A technician still provides machine-specific judgment, safety supervision, physical demonstrations, and practical experience that the system cannot fully reproduce. OperatorOS instead prepares the student so that their limited time with the technician can be used more effectively.

Our Solution

OperatorOS is a video-based learning and knowledge platform for manufacturing equipment.

A user can upload a machine video or provide a YouTube link. They can play the video, pause it at a useful moment, draw on the frame, and ask a question about what they are seeing. OperatorOS then combines several sources of information:

- The paused video frame.

- The user's question.
- Any marks or annotations drawn by the user.
- The video transcript around that moment.
- Previously asked questions in the conversation.
- Relevant passages from uploaded user manuals.

The system uses this combined context to answer the question and, when appropriate, visually point out the relevant area.

If the user asks OperatorOS to track an object, the system can also run SAM3 segmentation and tracking. Instead of only drawing a rough box, SAM3 identifies the visible pixels belonging to the object and creates a highlighted tracking video from the point where the user paused.

The goal is to make learning more visual, conversational, and connected to the machine being discussed.

What a Typical User Experience Looks Like

A normal session works like this:

1. The user uploads a local machine video or pastes a YouTube URL.
2. OperatorOS prepares the video, transcript, and supporting frame information.
3. The user plays the video and pauses when a relevant machine part is visible.
4. The user may draw a circle, arrow, rectangle, freehand mark, or text annotation over the frame.
5. The user asks a question such as:

``` `text What is this part?` ```

or:

``` `text What does this switch control?` ```

1. OperatorOS examines the paused frame, the user's annotations, the nearby transcript, and any selected manuals.
2. The system returns an explanation and may create visual annotations to support the answer.
3. If the user says:

``` `text Track this component.` ```

OperatorOS starts SAM3 and creates a new video with the selected object highlighted.

1. The user can play the processed tracking video normally.

This interaction allows the user to learn by pointing, asking, watching, and following up rather than reading a manual from beginning to end.
```

The Video Is the Main Source of Context

OperatorOS is designed as a video-first system.

This means the visible frame is treated as the most immediate source of truth. The transcript and uploaded manuals add useful background information, but the answer should remain grounded in what the user can actually see.

This decision is important because a transcript may mention something that is no longer visible, while a manual may describe several versions of a machine. If the visible frame and the supporting text appear to disagree, the system should explain the uncertainty instead of pretending that every source perfectly matches.

The video-first approach also makes the system more useful for visual questions, such as:

- Which part is the emergency stop?
- Where is the material loaded?
- What is connected to this cable?
- Which component is moving?
- Is this the same part mentioned in the manual?

Uploading a Local Video

The simplest way to begin is to upload a local video file.

OperatorOS stores a copy of the video and gives it a unique internal ID. This ID allows the different parts of the system to refer to the same video without repeatedly uploading it.

After receiving the file, the system:

1. Stores the original video.
2. Checks that it is a readable video.
3. Extracts speech or creates fallback timestamp information.
4. Extracts reference frames.
5. Stores video metadata such as the title.
6. Makes the video available to the browser through a seekable video stream.

The browser can then request only the part of the file that it needs. This is what allows the user to seek through the video without downloading the entire file again every time.

Downloading and Preparing YouTube Videos

YouTube support was one of the more difficult parts of the project.

Using an embedded YouTube player would have been much easier, but it would not have met the needs of OperatorOS. An embedded player does not give the rest of the system enough control over:

- Extracting the exact paused frame.
- Knowing the local path of the video.
- Processing the video with SAM3.
- Creating indexed frames.
- Running consistent transcription.
- Producing a new highlighted tracking video.

For these reasons, OperatorOS downloads a local working copy of the YouTube video.

The system uses `yt-dlp` and FFmpeg to download, combine, and verify the media. It also contains options for JavaScript challenge handling, retries, cookies, player types, network configuration, and other YouTube-specific issues.

YouTube changes its protections frequently. A method that works today may stop working after YouTube changes its player or anti-bot checks. Some videos also require login, cookies, or an IP address with a better reputation.

One of the main lessons from this work is that there is no universal cookie-free YouTube solution. Instead of hiding this reality, OperatorOS attempts to return useful guidance that explains whether the problem is likely related to:

- Login requirements.
- Rate limits.
- Cookies.
- JavaScript challenges.
- Missing software.
- Network restrictions.
- An unavailable video format.

Video Transcription

OperatorOS uses the transcript to understand what is being discussed around the paused moment.

For example, if a technician says:

> `This lever releases the material tray.`

while the lever is visible, the transcript provides useful context for a question asked at that timestamp.

The system uses Whisper to create timestamped speech segments. When the user asks a question, OperatorOS requests only the transcript window near the current video time rather than sending the full transcript every time.

This keeps the context more focused and reduces unnecessary information.

If Whisper is disabled or cannot run, OperatorOS can create fallback timestamp segments. These fallback segments allow the rest of the system to continue working, but they are not real speech transcription. The interface and API identify this condition so that a developer does not mistake fallback data for an accurate transcript.

Asking Questions About a Paused Frame

When the user sends a question, OperatorOS captures the current video frame as an image.

The question request contains:

- The current frame.
- The exact timestamp.
- The video title.
- The nearby transcript.
- The user's annotations.
- An optional image showing those annotations directly over the frame.
- Selected manual passages.
- Recent conversation history.

The system sends this information to a vision-language model through OpenRouter.

The vision-language model is expected to return:

1. A written answer.
2. Visual annotations that support the answer.
3. A short description of an object that should be tracked, when tracking is useful.
4. A precise tracking annotation, when the model can confidently locate the object.

The response is returned in a structured format so that OperatorOS can separate the written answer from the visual information.

Why We Send Annotation Coordinates

When a user draws on the video, OperatorOS records the annotation type and location.

For example, a rectangle may be represented by:

- Its horizontal position.
- Its vertical position.
- Its width.
- Its height.
- Its color.

All positions are normalized so that they remain meaningful even when the video is displayed at a different size.

Sending these coordinates helps the model understand which area the user is referring to. However, coordinates alone do not always communicate the user's intention clearly.

We found that a vision-language model may still misunderstand:

- Which side of a line is important.
- Whether a circle refers to the object inside it or something nearby.
- Whether a freehand mark covers one object or several objects.
- How the annotation visually relates to the original frame.

To improve this, OperatorOS includes an optional "Send Annotated Snapshot" setting.

When enabled, the system sends:

1. The clean original frame.
2. A second image with the user's drawings visibly placed over the frame.

The clean frame remains the main source image. The annotated image acts as visual guidance.

This improved consistency because the model could use its visual ability to see the same marks that the user sees, rather than reconstructing the meaning only from JSON coordinates.

Visual Answers and Model-Generated Annotations

OperatorOS can display annotations generated by the model.

These annotations can include:

- Rectangles.
- Circles.
- Arrows.
- Polygons.
- Freehand-style paths.
- Numbers.
- Text labels.

The purpose of these annotations is to visually support the answer. They are not intended to decorate the screen or cover unrelated areas.

The model is instructed to make annotations:

- Tight around the relevant object.
- Consistent across similar objects.
- Specific to what the answer discusses.
- Honest about uncertainty.

If the model cannot confidently locate something, it should omit the annotation rather than guess.

Conversational Memory

OperatorOS keeps a limited amount of recent conversation history.

This allows follow-up questions such as:

```
What does it connect to?
```

to make sense after the user previously asked:

```
What is this cable?
```

The system keeps only a rolling window of recent messages rather than an unlimited transcript. This provides enough context for follow-up questions without allowing the prompt to grow forever.

For local development, conversation state is stored in memory. Redis can be enabled for environments where several processes need to share the same state.

Learning From User Manuals

Users can upload manuals and supporting documents to OperatorOS.

The current document system is text-based. It:

1. Extracts readable text from the file.
2. Splits the text into overlapping sections called chunks.
3. Converts each chunk into a numerical representation called an embedding.
4. Stores those chunks and embeddings in a local index.

When the user asks a question, OperatorOS converts the question into an embedding and compares it with the stored manual chunks.

The most similar chunks are sent to the vision-language model as supporting evidence.

This process is called retrieval-augmented generation, or RAG.

The model is instructed to cite the filenames it uses and avoid inventing machine-specific instructions that are not present in the retrieved material.

Why Text-Based RAG Is Not Enough for Every Manual

The current RAG system works best when important information is written in text.

Manufacturing manuals often contain:

- Diagrams.
- Arrows.
- Numbered callouts.
- Photographs.
- Exploded views.
- Tables.
- Labels placed directly on images.
- Several views of the same machine.

Converting only the written text into embeddings leaves out much of this visual information.

This remains an open challenge.

Several possible approaches have been discussed:

1. Convert each manual page into both an image and a text representation.
2. Split each page into smaller visual regions.
3. Use CLIP-style image-text similarity to retrieve visually relevant pages.
4. Use a modern vision-language model to summarize each page during ingestion.
5. Store the page summary so the model does not repeatedly analyze the same page.
6. First retrieve a small number of likely pages, then run deeper visual analysis only on those pages.
7. Use page layout, headings, and nearby text to understand which machine component a diagram is explaining.

Each option has tradeoffs.

Running a large vision-language model on every page for every question would be slow and expensive. Running it once during ingestion and storing the result may be more efficient, but the stored interpretation must be detailed enough to support future questions.

Selecting only one highlighted component from a page may also create tunnel vision. A page may be explaining a larger procedure, even if one part is visually prominent.

Future work should therefore combine text, layout, and images rather than treating any single method as the complete solution.

How OperatorOS Decides to Start SAM3

SAM3 may start in several situations.

The clearest case is when the user explicitly asks for tracking:

```
Track this machine.
```

```
Follow the red component.
```

```
Monitor this lever.
```

```
Keep an eye on the moving belt.
```

OperatorOS recognizes direct tracking language such as:

- Track.
- Follow.
- Trace.
- Monitor.
- Keep an eye on.

An explicit request starts tracking even if the automatic tracking checkbox is disabled.

SAM3 may also start when:

- Automatic SAM3 tracking is enabled.
- The vision-language model determines that tracking would be useful.
- The model returns a tracking prompt or a tracking annotation.

The VLM Does Not Need to Draw a Box First

An earlier version of the system required the vision-language model to return a bounding box before SAM3 could start.

This caused a frustrating failure:

```
No trackable target returned by VLM.
```

The user may have clearly said:

```
Track the machine.
```

but the system still refused because no box was returned.

We learned that a bounding box is useful but should not be mandatory.

The current approach is:

1. Use a precise tracking annotation when the model returns one.
2. Otherwise, use a short text description such as `the large black machine`.
3. If necessary, use the user's original tracking request.

SAM3 can therefore run from either a visual box or a text prompt.

How SAM3 Segmentation and Tracking Work

SAM3 does more than draw a rectangle around an object.

It creates a segmentation mask that identifies the pixels belonging to the object. A mask can follow the shape of the object more closely than a simple bounding box.

OperatorOS uses two Ultralytics SAM3 modes:

- A box-prompted mode when a reliable object location is available.
- A text-prompted mode when the system only has a description.

The model uses a local `sam3.pt` checkpoint and runs on the RTX 4090 through CUDA.

Text-prompted SAM3 required the Ultralytics version of CLIP. We originally installed OpenAI's CLIP repository, but that version used an incompatible tokenizer interface and produced this error:

```
'SimpleTokenizer' object is not callable
```

Switching to the Ultralytics CLIP repository fixed semantic text tracking.

Why the First Tracking Overlay Failed

Our first plan was to keep playing the original video and draw SAM3 polygons over it in the browser.

This approach seemed attractive because the user could continue watching the same video while the mask moved over the object.

In practice, it introduced several problems.

The Mask Appeared in the Upper-Left Corner

SAM3 coordinates used a `0–100` percentage scale, while other annotations used a `0–1000` scale.

The browser mistakenly scaled the SAM3 coordinates a second time. This compressed the mask into the upper-left corner.

The lesson was that every coordinate format must have a clearly defined scale.

The Mask Became a Crisscrossed Shape

Some SAM3 masks contain several disconnected regions.

The original conversion treated all regions as one polygon. The browser connected the end of one region to the beginning of another with straight lines, creating a web of diagonal lines.

We later used the binary mask data and extracted each connected contour separately. This fixed the polygon shape, but other timing and performance problems remained.

The Mask Blinked

The first implementation processed only one frame per second.

The mask appeared on a processed frame, disappeared between frames, and appeared again one second later.

This was not smooth tracking. It was a sequence of isolated segmentation results.

We changed SAM3 to process consecutive frames at the original video's frame rate.

For a video running at approximately 60 frames per second, a new processed frame is created roughly every 0.0167 seconds.

Playback Outran SAM3

Even after using the correct frame rate, the browser could play the video faster than SAM3 could generate and deliver mask coordinates.

When playback moved beyond the latest processed frame, the mask disappeared.

The system could try to hold the previous mask, but that mask might no longer match the object's new position.

The Coordinate Timeline Became Too Large

A long video at 60 frames per second may contain thousands of masks. Each mask may contain hundreds of points.

Sending this complete coordinate timeline to the browser created unnecessary complexity:

- Large messages.
- Repeated serialization.

- Event timing problems.
- Browser rendering work.
- Synchronization problems.
- Difficult error recovery.

This led to a major design change.

The Current SAM3 Approach: Generate a New Tracking Video

The current approach follows the behavior of a standalone SAM3 script that was already working correctly.

Instead of asking the browser to rebuild every mask, OperatorOS creates a new processed video.

The workflow is:

1. The user pauses the original video.
2. OperatorOS records the exact source timestamp.
3. The video is clipped from that timestamp to the end.
4. SAM3 processes every consecutive frame.
5. The mask is blended directly into the pixels of each video frame.
6. The processed frames are written into a new video.
7. The video is converted into a browser-compatible format.
8. The frontend replaces the original player source with the processed clip.
9. The user presses play and watches normal video playback with the SAM3 highlight already included.

This approach removes the need to synchronize browser polygons with a video that is still playing.

How the SAM3 Highlight Is Created

For each processed frame:

1. SAM3 returns the original image and one or more binary masks.
2. Each binary mask identifies the pixels considered part of the object.
3. The mask is resized to match the original frame when necessary.
4. A translucent green color is blended into the masked pixels.
5. The highlighted frame is written into the output video.

The result resembles the filled segmentation examples shown in official SAM3 demonstrations.

The mask is not merely an outline. The object itself is visibly highlighted.

Why the Tracking Video Needed Another Conversion

OpenCV initially created an MP4 using the `mp4v` codec.

The file extension was `.mp4`, but the browser still rejected it with an unsupported-stream error.

This taught us that an MP4 file is only a container. Browser compatibility also depends on the video codec and pixel format inside the container.

OperatorOS now uses FFmpeg to convert the intermediate video into:

- H.264 video.
- YUV420p pixel format.
- A fast-start MP4.

Fast-start moves important playback metadata to the beginning of the file so the browser can begin loading it correctly.

The generated video also supports HTTP range requests, allowing the user to seek through it.

Processing From the Paused Point to the End

The generated tracking video begins where the user paused the original video.

For example:

- The original video is paused at 12 seconds.
- SAM3 creates a new clip beginning at original time 12 seconds.
- Time `0:00` in the processed clip corresponds to time `0:12` in the original.

OperatorOS remembers this offset.

If the user later asks a question at processed-video time `0:05`, the system knows that the real source time is `0:17`.

This allows transcript retrieval and later questions to remain aligned with the original video.

The current system does not yet join the unchanged first 12 seconds with the processed remainder. It switches the player to a new clip beginning at the tracking point.

A future version could:

1. Keep the original prefix.
2. Add the processed tracking section after it.
3. Preserve the original audio.
4. Produce one complete combined video.

Why Some Generated Videos Lasted Less Than One Second

At one stage, the generated result looked correct but lasted only about one third of a second.

ffprobe showed:

```
20 frames
approximately 60 frames per second
approximately 0.333 seconds
```

The system had previously used a 20-frame testing limit.

We changed the configuration so:

```
SAM3_MAX_PROPAGATION_FRAMES=0
```

means “process every remaining frame.”

However, an older VS Code terminal still contained the value `20`. Python treated that inherited value as more important than the updated `.env` file.

Restarting from the same terminal continued to produce 20-frame clips.

The root launcher now reads the repository `.env` and applies it over stale terminal variables whenever the services start.

The SAM3 health endpoint also reports the effective frame limit so this problem can be seen directly.

Tracking Progress and Generated Video Playback

Full-video SAM3 processing can take time, especially at a high frame rate and 1024-pixel inference resolution.

While the job is running:

- OperatorOS pauses the original video.
- The interface displays the job ID.
- Progress updates are sent to the browser.
- The generated file remains unavailable until processing and encoding finish.

When the job completes:

- OperatorOS receives the final rendered-video location.
- The player switches to the new tracking clip.
- A cache-busting value is added to prevent the browser from reusing an older failed response.
- The user can play and seek through the completed tracking result.

Why OperatorOS Uses Several Services

OperatorOS is divided into separate services because each major task has different requirements.

Frontend

The frontend manages:

- The video player.
- Drawing tools.
- Chat.
- Document selection.
- Tracking controls.
- Progress and error messages.
- Switching to generated tracking videos.

Orchestrator

The orchestrator is the central traffic controller.

It:

- Receives requests from the frontend.
- Calls the correct backend service.
- Combines retrieved information.
- Stores recent conversation history.
- Starts tracking jobs.
- Proxies generated videos.
- Streams responses and progress.

Video Service

The video service handles:

- Local uploads.
- YouTube downloads.
- Video validation.
- Transcription.
- Frame extraction.
- Video metadata.
- Seekable source-video delivery.

RAGVLM Service

The RAGVLM service handles:

- Prompt construction.
- Vision-language model requests.
- Document extraction.
- Document chunking.
- Embeddings.
- Retrieval.
- Structured visual answers.

SAM3 Service

The SAM3 service handles:

- Model and checkpoint loading.
- Box and text prompts.
- Video clipping.
- Frame-by-frame mask generation.
- Highlight rendering.
- H.264 output encoding.
- Tracking status.
- Generated-video delivery.

Keeping these responsibilities separate makes it easier to test and debug one part without running every feature inside one very large application.

Why Redis Is Optional

The early project expected Redis to be running for local development.

This created unnecessary setup problems. Importing the application could fail before a developer even reached the feature they wanted to test.

The current local workflow stores:

- Conversation state.
- Tracking job state.

in memory by default.

Redis and the background worker remain available for a deployment that needs shared state or queued jobs, but they are not required for the normal local demo.

Error Reporting and Health Checks

One major lesson from debugging was that generic error messages slow down development.

For example:

```
Tracking failed to start.
```

did not reveal whether the real problem was:

- A missing model.
- A wrong service URL.
- A CPU-only PyTorch installation.
- An incompatible CLIP package.
- A missing video.
- A failed FFmpeg conversion.
- A stale environment variable.

OperatorOS now tries to display the backend error when a request fails.

The SAM3 health endpoint reports:

- Whether the backend is ready.
- Whether the checkpoint exists.
- The resolved checkpoint path.
- PyTorch version.
- CUDA version.
- Whether CUDA is available.
- GPU name.
- Effective frame limit.
- Effective inference size.
- Whether simulation is enabled.

These values are useful because they describe what the running process is actually using, not only what a configuration file claims it should use.

What We Verified on the RTX 4090

Real GPU validation included:

- Loading the local `sam3.pt` checkpoint.
- Detecting the RTX 4090.

- Running CUDA-enabled PyTorch.
- Running box-prompted SAM3.
- Running text-prompted SAM3.
- Processing consecutive frames.
- Reading binary mask data.
- Creating highlighted frames.
- Writing a processed video.
- Converting the result to H.264.
- Confirming the frame rate and frame count.
- Confirming that the browser endpoint supports seeking.

The automated Python test suite currently contains 43 tests, and the frontend is checked through a production build and TypeScript validation.

Current Limitations

OperatorOS is functional, but it is still a development system.

Processing Time

Processing every remaining frame at high resolution can be slow.

Possible future improvements include:

- Quality and speed presets.
- A user-selected tracking duration.
- Time estimates.
- Faster hardware encoding.
- Persistent background jobs.

Audio

The generated tracking clip currently does not preserve the original audio.

Generated File Cleanup

Processed videos are stored locally. Automatic cleanup and expiration have not yet been implemented.

One Highlight Color

The current processed result uses a green highlight. Multiple stable colors could be added when distinguishing several object instances becomes important.

Returning to the Original Video

The current interface replaces the player with the processed clip. A clearer control for returning to the original video would improve the experience.

Diagram-Heavy Manuals

Text RAG does not yet understand diagrams as well as it understands written paragraphs.

Live Camera Use

The system currently works with uploaded or downloaded videos. The long-term goal includes live camera understanding, voice interaction, and mobile use, but those features are not yet implemented.

End Goal

The long-term vision is for OperatorOS to become a mobile, voice-enabled machine-learning assistant.

A user could point a phone camera at a machine and ask:

> `What is this machine?`

> `How do I begin the startup procedure?`

> `Which switch should I press next?`

> `Show me the emergency stop.`

The system could:

- Understand the live camera view.
- Highlight the relevant machine component.
- Read instructions aloud.
- Listen to spoken questions.
- Retrieve the correct manual information.
- Cite the source.
- Guide the user through a sequence of steps.
- Track the selected component as the camera or machine moves.

The desired experience is similar to a conversational voice assistant combined with visual annotations, machine-specific retrieval, segmentation, and tracking.

This end goal requires more work in:

- Safety and uncertainty handling.
- Live video processing.
- Voice input and speech output.
- Diagram-aware retrieval.
- Reliable step-by-step procedure generation.
- Mobile performance.
- Human technician review and validation.

OperatorOS should continue to be presented as a learning aid, not as an unsupervised replacement for qualified machine training.

Main Lessons From the Project So Far

1. A model running successfully does not guarantee that the user-facing feature works.

Coordinates, timing, video codecs, browser support, and service configuration must all be tested together.

1. Standalone working code is extremely valuable.

The successful SAM3 standalone script helped us stop guessing and align the integrated pipeline with a known-good result.

1. Visual problems should be solved visually.

Annotation JSON was useful, but sending an annotated image improved the model's understanding.

1. Tracking must match the video frame rate.

One result per second cannot look like smooth tracking in a 60-frame-per-second video.

1. A browser overlay is not always the simplest solution.

Baking masks into a processed video proved more reliable than streaming thousands of polygons.

1. File extensions do not guarantee browser compatibility.

The final codec, pixel format, metadata, and range support all matter.

1. Runtime configuration must be visible.

A stale terminal variable can override the correct `.env` value and create results that appear mysterious.

1. Simulation must never be mistaken for real model output.

Real demos keep simulation disabled.

1. The system should explain uncertainty.

This applies to VLM answers, missing transcript data, document retrieval, YouTube failures, and machine-specific safety information.

1. OperatorOS is strongest when its features work together.

The main value comes from connecting video, visual questions, annotations, transcripts, manuals, conversation, segmentation, and tracking into one learning experience.